

《信息安全综合实验》实验报告

实验 1: 私有 CA 证书签发的简单实现

姓名: 蒋桢言 学号: 2313546

1 实验内容及要求

深入理解 PKI 标准的概念, 以及其应用 SSL 协议的实现流程, 掌握 OpenSSL 工具的使用方法, 完成私有 CA 搭建及证书签发和吊销的实践。

1. 熟悉 Linux 环境下 OpenSSL 工具的使用。
2. 搭建私有 CA 并生成 CA 根证书。
3. 模拟终端实体向 CA 发送请求, 完成证书签发、吊销流程的简单实现。

2 实验过程

2.1 实验环境

本次实验使用的操作系统为 Ubuntu 24.04, 需要用到的工具是 OpenSSL。执行 `openssl version` 命令, 发现输出了版本号, 说明 OpenSSL 已经安装。

```
jry@jry-VMware-Virtual-Platform:~/桌面/lab1$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

图 1: OpenSSL 版本

2.2 搭建私有 CA

首先创建私有 CA 所需要的文件目录, 保存 CA 的相关信息。依次执行以下命令:

```
bash

mkdir myCA // 创建CA根文件夹
cd myCA // 进入CA根文件夹
mkdir newcerts private conf // 创建三个文件夹, 用来存放新发放证书、私
    钥和配置文件
chmod g-rwx,o-rwx private // 设置private文件夹的操作权限
touch index serial crlnumber // 创建证书信息数据库、证书序号文件、crl序
    号文件
echo 01 > serial // 初始化证书的序号
echo 01 > crlnumber // 初始化吊销证书序号
```

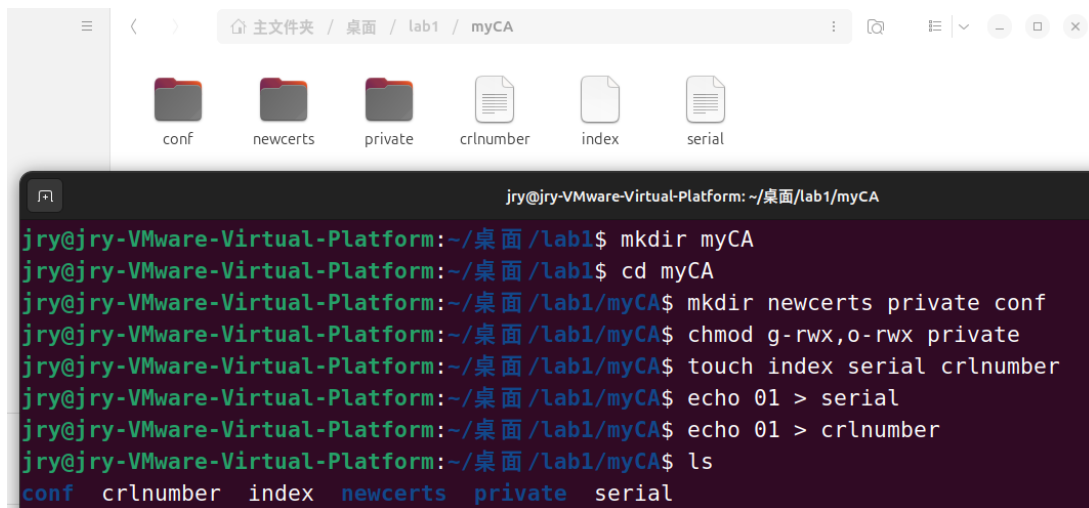


图 2: 创建私有 CA 所需要的文件目录

然后创建生成 CA 自签名证书的配置文件。依次执行以下命令：

```
bash
```

```
cd conf          //进入配置文件夹  
vim genca.conf   //创建用来生成自签名证书的配置文件
```

将 genca.conf 修改为以下内容：

```
ini
```

```
[ req ]  
default_keyfile = /home/jry/lab1/myCA/private/cakey.pem  
default_md = md5  
prompt = no  
distinguished_name = ca_distinguished_name  
x509_extensions = ca_extensions  
  
[ ca_distinguished_name ]  
organizationName = JRYorg  
organizationalUnitName = JRYunit  
commonName = JRY  
emailAddress = 2313546@mail.nankai.edu.cn  
  
[ ca_extensions ]  
basicConstraints = CA:true
```

内容分析：

(1) [req] 部分——证书请求配置，这是 OpenSSL req 命令的配置参数。

default_keyfile = /home/jry/lab1/myCA/private/cakey.pem 是默认使用的 CA 私钥

的文件路径。

`default_md = md5`意思是默认签名哈希算法是 MD5。

`prompt = no`意思是不要交互输入证书信息, 直接从 `openssl.cnf` 中读取。

`distinguished_name = ca_distinguished_name`意思是使用 `[ ca_distinguished_name ]` 作为证书主体信息, 也就是证书里的 Subject 来自这个 section。

`x509_extensions = ca_extensions`意思是生成 X.509 证书时使用 `[ ca_extensions ]` 扩展字段, 即证书里 X509v3 extensions 来自这个 section。

(2) `[ ca_distinguished_name ]` 部分——证书主体信息, 这是证书里的 Subject 结构。

`organizationName = JRYorg`是组织名称。

`organizationalUnitName = JRYunit`是部门。

`commonName = JRY`是 CA 名称。

`emailAddress = 2313546@mail.nankai.edu.cn`是邮箱。

(3) `[ ca_extensions ]` 部分——CA 证书扩展字段。

`basicConstraints = CA:true`用于决定证书是否可以作为 cA 证书使用, 此处设置为 `true`。

最后生成私有 CA 的私钥和自签名证书 (根证书)。执行命令:

```
bash
```

```
openssl req -x509 -newkey rsa:2048 -out cacert.pem -outform PEM -days  
2190 -config /home/jry/lab1/myCA/conf/genca.conf
```

命令含义:

`openssl req`是 OpenSSL 生成证书签名请求的命令。

`-x509`表示生成自签名 X.509 证书。

`-newkey rsa:2048`表示生成一个新的 RSA 密钥对, 长度为 2048 位。

`-out cacert.pem`指定生成的证书文件输出路径, 这里输出的证书文件是 `cacert.pem`。

`-outform PEM`指定证书的编码格式为 PEM。PEM 是 ASCII 编码格式, 常用 `—BEGIN CERTIFICATE—`包裹。

`-days 2190`指证书有效期为 2190 天。

`-config /home/jry/lab1/myCA/conf/genca.conf`指定 OpenSSL 配置文件。

执行后要求输入 CA 私钥的保护密码, 这里设置为 `myCA2026`。确认后如图3所示。



图 3: 生成私有 CA 的私钥和自签名证书

当前目录下得到 cacert.pem (CA 私钥), private 目录下得到 cakey.pem (CA 证书)。

这时直接执行 `cat private/cakey.pem` 会得到如图4所示的内容，这是加密之后的 CA 私钥再进行 base64 编码得到的内容。如果想要查看 CA 私钥的明文，需要执行如下命令，并输入先前设置的 CA 私钥的保护密码（myCA2026），结果如图5所示。

bash

```
openssl pkey -in private/cakey.pem -text -noout
```

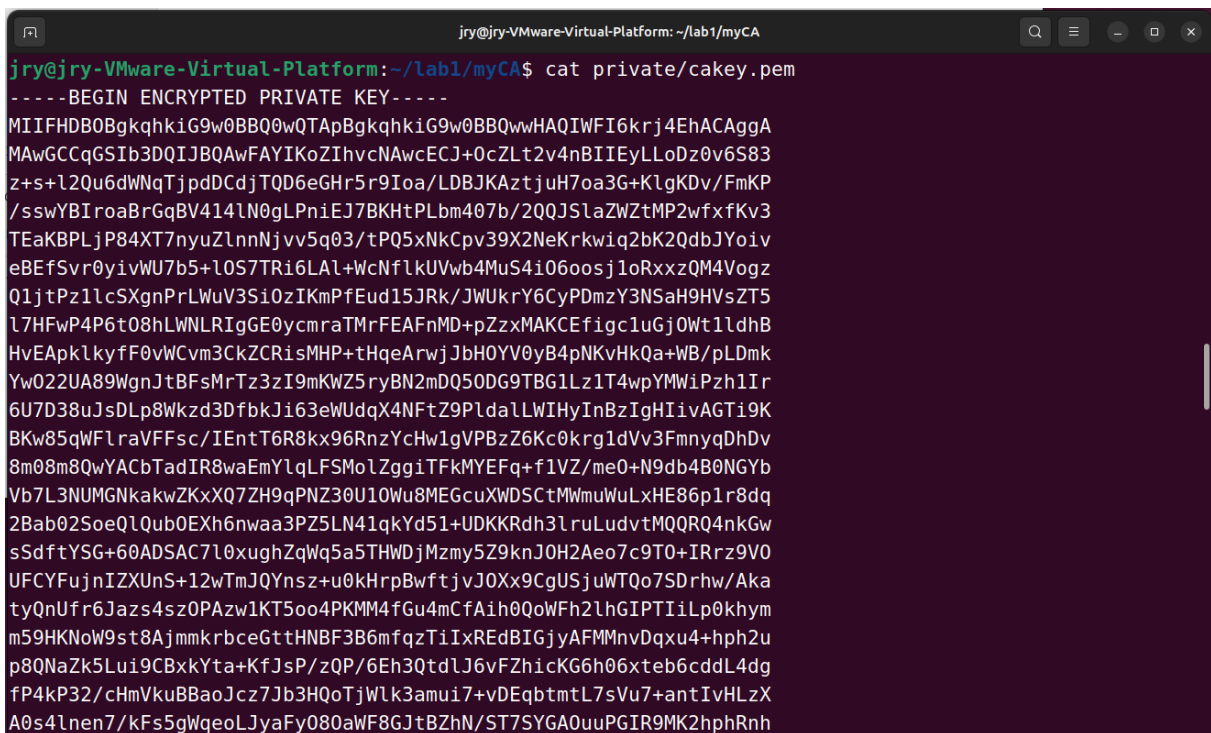


图 4: 加密并 base64 编码之后的 CA 私钥

```
jry@jry-VMware-Virtual-Platform: ~/lab1/myCA
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ openssl pkey -in private/cakey.pem -text -noout
Enter pass phrase for private/cakey.pem:
Private-Key: (2048 bit, 2 primes)
modulus:
    00:95:17:cc:d4:f8:b0:31:2d:8f:93:04:11:c7:27:
    c4:32:1b:b1:6e:83:3e:a5:b4:0b:dd:bb:40:ad:41:
    a1:bc:fc:50:9d:b2:19:f1:74:69:a0:da:e4:9e:bc:
    5c:57:96:b5:92:ff:27:26:dc:24:9d:f8:39:ee:77:
    e3:37:e5:ea:da:a5:db:7f:64:da:ee:0b:c2:a0:de:
    93:e4:f8:81:00:3c:e9:be:e9:97:78:64:76:d9:b4:
    08:d0:15:91:c2:a9:eb:9c:6d:25:0f:42:43:0f:e2:
    1a:2a:76:e9:36:ec:3a:30:79:2f:f7:ca:f3:fd:75:
    ce:d2:1e:6f:fa:32:f9:d9:88:ac:80:bc:76:6f:b0:
    05:22:ea:93:77:e9:07:6f:ec:32:e6:8a:64:6d:cf:
    a3:52:5c:ea:07:41:50:d5:ca:68:f2:14:d9:71:93:
    8d:5f:9b:52:e0:69:8d:20:21:4f:b7:4b:c9:93:d8:
    12:73:86:e3:1e:86:2b:52:fa:6a:72:53:aa:e3:8b:
    c3:3b:28:79:97:0d:e0:1e:8b:a7:48:30:72:61:1f:
    35:0d:f4:5d:76:ab:60:da:00:33:77:12:50:de:2f:
    7e:3c:51:0e:36:98:83:9c:ac:a1:be:e9:46:09:e1:
    7c:6f:b8:f9:15:6c:54:4f:e7:a7:55:c0:a9:ef:98:
    43:5f
publicExponent: 65537 (0x10001)
privateExponent:
```

图 5: CA 私钥

再执行以下命令查看 CA 自签名证书，如图6所示：

```
bash

openssl x509 -in cacert.pem -text -noout
```

```
jry@jry-VMware-Virtual-Platform: ~/lab1/myCA
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ openssl x509 -in cacert.pem -text -noout
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            23:f3:95:30:79:b0:01:f1:97:98:5b:e0:fc:c4:04:ef:66:55:ce:47
        Signature Algorithm: md5WithRSAEncryption
        Issuer: O = JRYorg, OU = JRYunit, CN = JRY, emailAddress = 2313546@mail.nankai.edu.cn
        Validity
            Not Before: Mar  4 11:32:58 2026 GMT
            Not After : Mar  2 11:32:58 2032 GMT
        Subject: O = JRYorg, OU = JRYunit, CN = JRY, emailAddress = 2313546@mail.nankai.edu.cn
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (2048 bit)
                Modulus:
                    00:95:17:cc:d4:f8:b0:31:2d:8f:93:04:11:c7:27:
                    c4:32:1b:b1:6e:83:3e:a5:b4:0b:dd:bb:40:ad:41:
                    a1:bc:fc:50:9d:b2:19:f1:74:69:a0:da:e4:9e:bc:
                    5c:57:96:b5:92:ff:27:26:dc:24:9d:f8:39:ee:77:
```

图 6: 查看 CA 证书

直接点开 cacert.pem 如图7所示。

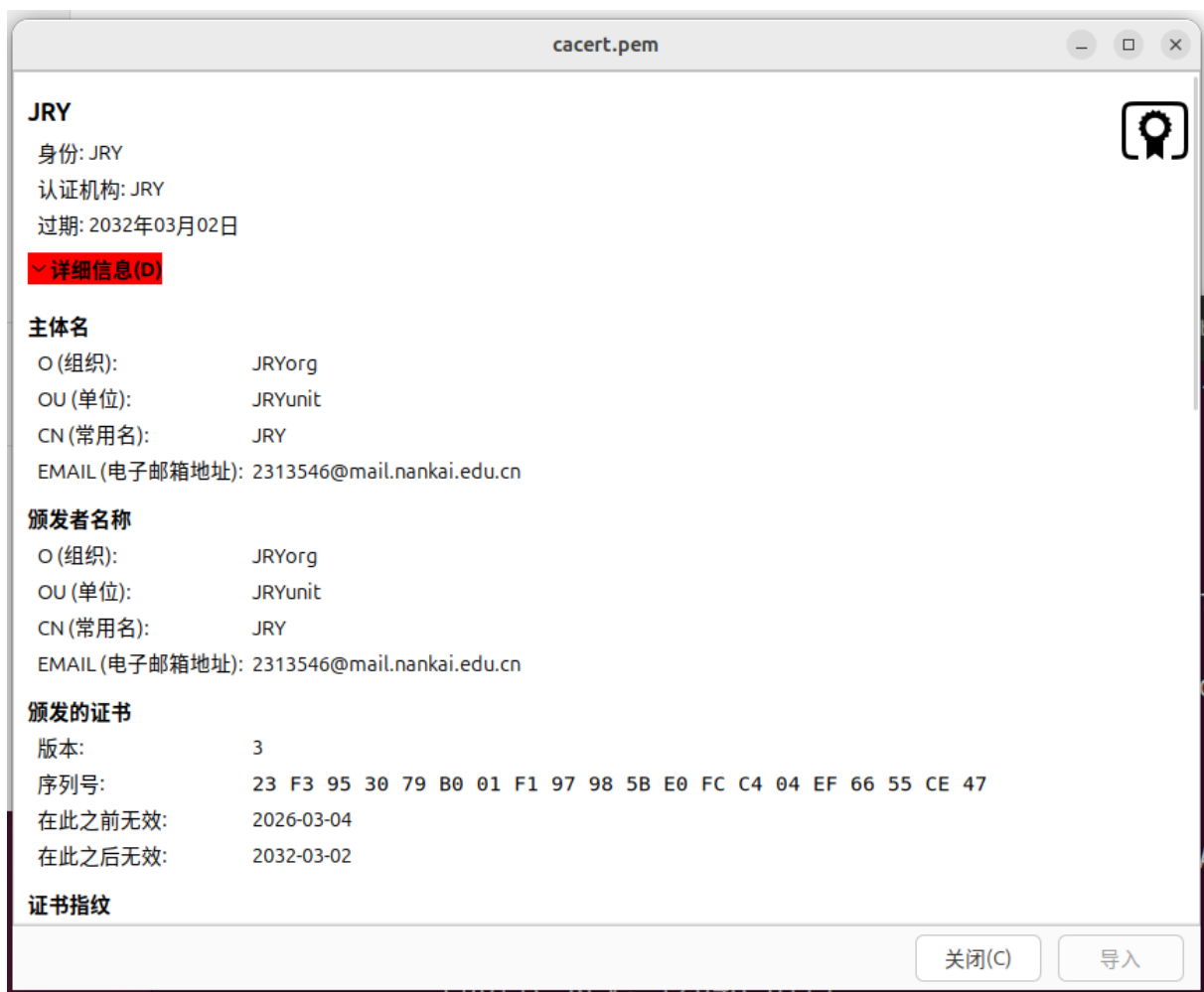


图 7: 直接查看 CA 证书

2.3 私有 CA 为服务器签发证书

首先创建用来为其他请求签发证书的配置文件。依次执行以下命令：

```
bash

cd conf          //再次进入配置文件夹
vim ca.conf      //创建用来为其他请求签发证书的配置文件
```

将 ca.conf 修改为以下内容：

```
ini

[ ca ]
# 指定默认CA配置段是myCA
default_ca = myCA
```

```

[ myCA ]
# CA根目录，后面的$dir都指这个目录
dir = /home/jry/lab1/myCA
# CRL文件存放目录
crl_dir = $dir/conf
# 证书数据库文件
database = $dir/index
# 新证书目录
new_certs_dir = $dir/newcerts

# CA证书文件
certificate = $dir/cacert.pem
# 证书序列号计数器
serial = $dir/serial
# CA私钥位置
private_key = $dir/private/cakey.pem
# CRL序列号文件
crlnumber = $dir/crlnumber
# 随机种子文件
RANDFILE = $dir/private/.rand

# 证书默认有效期365天
default_days = 365
# CRL有效期30天
default_crl_days = 30
# 签名使用MD5
default_md = md5
# 允许多个证书有相同（没有唯一性）
unique_subject = no

# 使用[policy_any]作为证书策略
policy = policy_any

[ policy_any ]
# optional表示可以有也可以没有，supplied表示必须提供
stateOrProvinceName = optional
localityName = optional
organizationName = optional
organizationalUnitName = optional
commonName = supplied
emailAddress = optional

```

然后模拟服务器，生成私钥与证书申请的请求文件。依次执行以下命令：

```
bash
```

```
mkdir server //创建服务器文件夹server
cd server
openssl req -newkey rsa:1024 -keyout server.key -out serverreq.pem -
    subj "/O=ServerCom/OU=ServerOU/CN=server"
//生成server的1024位私钥server.key和证书申请的请求文件serverreq.pem,
    此时需要设置服务器的私钥保护密码
```

服务器的私钥保护密码设置为 `myServer2026`，执行后生成服务器私钥（server.key 文件）和证书签名请求（CSR）（serverreq.pem 文件），如图8所示。



图 8: 生成私钥与证书申请的请求文件

最后 CA 根据服务器的证书请求文件生成证书并将其返回给服务器。执行以下命令：

```
bash
```

```
openssl ca -in serverreq.pem -out server.crt -config /home/jry/lab1/
    myCA/conf/ca.conf
//向私有CA提交证书请求文件serverreq.pem，CA生成并返回证书server.crt
//生成证书的规则是参照之前为CA定义的ca.conf配置文件执行的
```

过程中会提示输入 CA 私钥密码（之前设置的 myCA2026），并询问是否签名（y），是否提交（y）。成功后会在当前目录生成 server.crt，同时在/home/jry/lab1/myCA/newcerts 下保存一份副本（以序列号命名，01.pem），如图9所示。

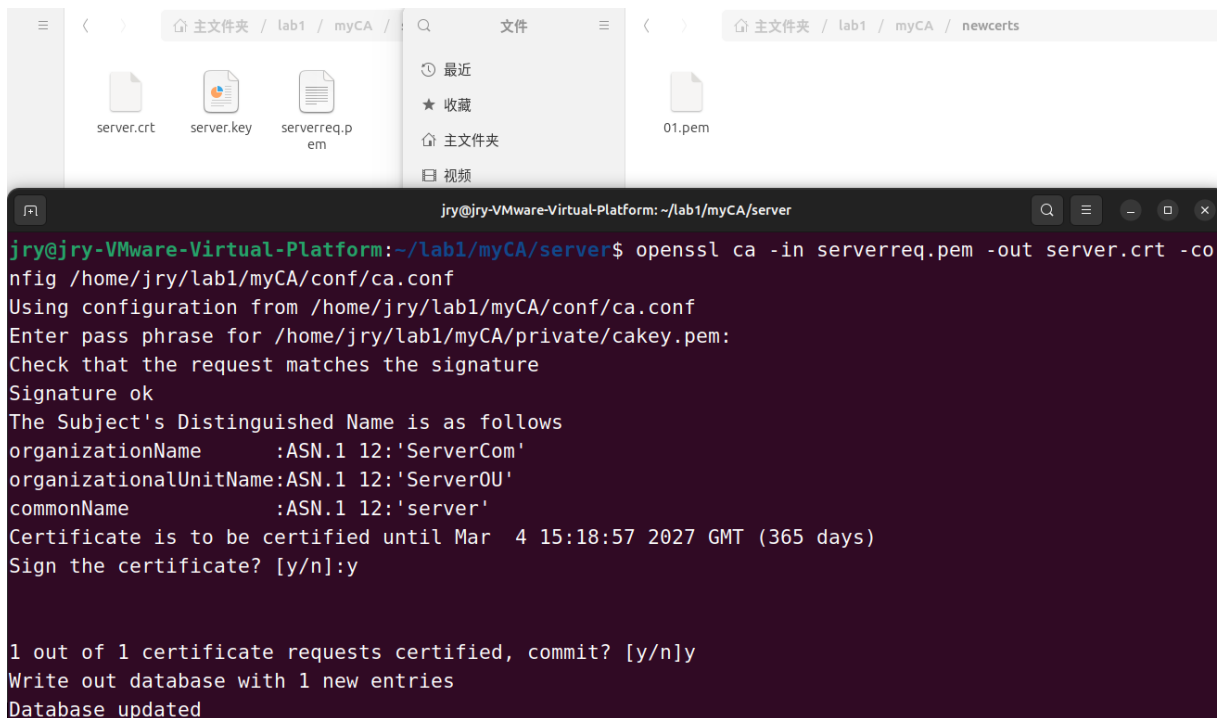


图 9: 生成证书

最后执行 `cat index` 发现证书信息数据库已更新，如图10所示。

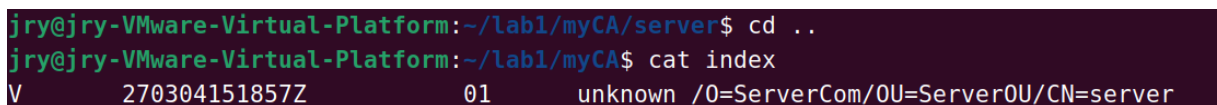


图 10: 证书信息数据库 index

2.4 私有 CA 为客户端签发证书

私有 CA 为客户端签发证书的过程和为服务器签发证书基本一致，依次执行以下命令：



同样输入 CA 密码并确认，如图11所示。

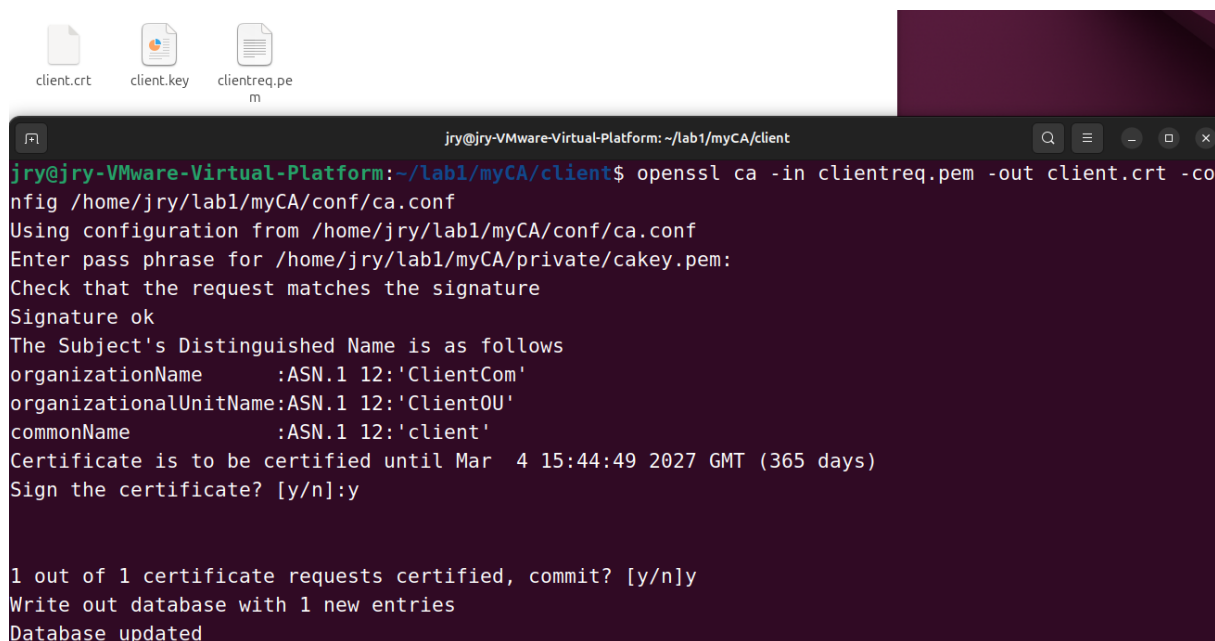


图 11: CA 为客户端签发证书

2.5 CA 吊销用户证书

使用命令 `cat index` 查看所有已签发证书及其状态, 刚刚已经查看过一次, 再次查看如图12所示。

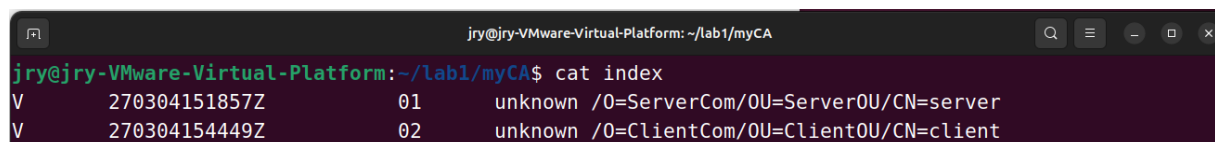
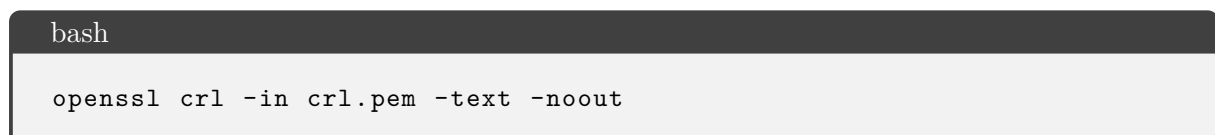


图 12: 查看所有已签发证书

执行以下命令吊销刚刚给服务器签发的证书 (01.pem) 并更新 (生成) 证书吊销列表 (CRL):



生成的 `crl.pem` 即证书吊销列表, 如图13所示。执行以下命令查看其内容, 如图14所示:



```
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ openssl ca -revoke /home/jry/lab1/myCA/newcerts/01.pem
-config /home/jry/lab1/myCA/conf/ca.conf
Using configuration from /home/jry/lab1/myCA/conf/ca.conf
Enter pass phrase for /home/jry/lab1/myCA/private/akey.pem:
Revoking Certificate 01.
Database updated
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ openssl ca -gencrl -out crl.pem -config /home/jry/lab1/myCA/conf/ca.conf
Using configuration from /home/jry/lab1/myCA/conf/ca.conf
Enter pass phrase for /home/jry/lab1/myCA/private/akey.pem:
```

图 13: 吊销证书并更新 CRL

```
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ openssl crl -in crl.pem -text -noout
Certificate Revocation List (CRL):
  Version 2 (0x1)
  Signature Algorithm: md5WithRSAEncryption
  Issuer: O = JRYorg, OU = JRYunit, CN = JRY, emailAddress = 2313546@mail.nankai.edu.cn
  Last Update: Mar  4 16:01:27 2026 GMT
  Next Update: Apr  3 16:01:27 2026 GMT
  CRL extensions:
    X509v3 CRL Number:
      1
Revoked Certificates:
  Serial Number: 01
    Revocation Date: Mar  4 15:52:27 2026 GMT
  Signature Algorithm: md5WithRSAEncryption
  Signature Value:
    67:23:09:0e:6b:e5:f4:8e:09:5f:74:80:4f:20:19:a0:73:39:
    27:51:4d:b2:c5:b5:fb:ee:f5:79:22:ea:e6:88:b3:bc:6d:fd:
    9a:bf:53:ea:da:ef:c1:f0:71:7b:22:23:9b:ca:8b:31:c6:75:
    6a:64:fb:a7:f5:77:cf:83:ad:83:74:4c:ef:54:31:d0:93:76:
    ae:52:90:eb:d5:e9:31:8a:30:ee:3b:3e:2c:26:99:16:af:14:
    8a:f7:b7:89:9c:80:b3:4c:50:9d:07:ac:e9:44:4a:82:db:e0:
    45:2b:25:7a:e8:50:c8:c4:3e:03:98:35:7c:35:89:f5:eb:5d:
    4e:3d:25:cd:41:83:4f:2d:8a:3a:7e:1d:c9:98:2c:9c:f0:eb:
    37:c2:45:a4:c3:0e:99:70:42:6c:cd:60:b4:76:a9:ea:68:bd:
```

图 14: 查看 CRL

再次命令 `cat index`，发现序列号为 01 的证书已被吊销，如图15所示。其中 V 表示有效 (Valid)，R 表示已吊销 (Revoked)，E 表示已过期 (Expired)，第一行 270304151857Z 为证书到期时间，260304155227Z 为证书吊销时间（仅吊销行有）。

```
jry@jry-VMware-Virtual-Platform:~/lab1/myCA$ cat index
R      270304151857Z    260304155227Z    01      unknown /O=ServerCom/OU=ServerOU/CN=server
V      270304154449Z           02      unknown /O=ClientCom/OU=ClientOU/CN=client
```

图 15: 再次查看所有已签发证书

至此就完成了私有 CA 证书签发并吊销的流程。

2.6 文件结构

本次实验的文件结构如下：

```

myCA/ ..... CA 根目录
├── conf/ ..... 配置文件目录
│   ├── genca.conf ..... 生成 CA 自签名证书的配置文件
│   └── ca.conf ..... CA 签发证书的配置文件
├── private/ ..... 存放 CA 私钥目录
│   └── cakey.pem ..... CA 加密私钥（已加密）
├── newcerts/ ..... 存放已签发证书副本的目录
│   ├── 01.pem ..... 第一个签发的证书（服务器证书）
│   └── 02.pem ..... 第二个签发的证书（客户端证书）
├── cacert.pem ..... CA 自签名根证书（公钥证书）
├── index ..... 证书数据库文本文件（记录已签发证书状态）
├── serial ..... 当前证书序列号（初始为 01）
├── crlnumber ..... 当前 CRL 序列号（初始为 01）
├── crl.pem ..... 生成的证书吊销列表（CRL）
├── server/ ..... 服务器证书申请目录（其实不用放在 myCA 文件夹下）
│   ├── server.key ..... 服务器私钥（已加密）
│   ├── serverreq.pem ..... 服务器证书请求文件（CSR）
│   └── server.crt ..... CA 签发的服务器证书
├── client/ ..... 客户端证书申请目录（也不用放在 myCA 文件夹下）
│   ├── client.key ..... 客户端私钥（已加密）
│   ├── clientreq.pem ..... 客户端证书请求文件（CSR）
│   └── client.crt ..... CA 签发的客户端证书

```

3 思考题

问：CA 如何验证证书的有效性？需要考虑到哪些方面？

答：主要验证 4 部分。

①**数字签名验证**。使用 CA 公钥验证证书上的签名是否与证书的哈希值一致，若一致则说明证书未被篡改且确实由该 CA 签发。

②**有效期检查**。检查当前时间是否在证书的有效期内。

③**吊销状态查询**。通过 CRL 或 OCSP 检查证书是否已被吊销。

④**域名/主体匹配**。验证证书中的 CN 或 SAN 扩展是否与当前访问的域名完全匹配（包括通配符证书的匹配规则）。

以上需要从该证书向上追溯至根证书，确保路径中的每一级证书都有效、未过期、未被吊销，且最终根证书存在于依赖方的受信任存储区。

除此之外，还可能包括算法安全性检查，证书使用的签名哈希算法应足够安全（如 SHA-256），避免使用已知弱算法（如 MD5、SHA-1）。